

MVP Testing Direction Document (Frontend Only)

Project: Castglo

Scope: Frontend

Generated: 2026-05-28

Scope note: This document focuses on frontend testing only (UI, routing, client-side state, browser behavior, and frontend integration contracts). Backend/database behavior is referenced only where the frontend depends on API response shapes and error handling.

1. Product Overview

What the application does (frontend-visible behavior)

Castglo is a multi-role web application that enables:

- Public marketing + discovery (landing, about/contact, news, browse talent, browse castings, public profiles).
- Role-based authenticated dashboards:
 - Talent:** browse castings, view details, submit auditions, manage submissions, messages, notifications, profile, account settings, livestream/audition experiences.
 - Casting Director:** create/manage casting calls/projects, review submissions/applicants pipeline, messaging, collaborators, matched talent, billing.
 - Industry Professional:** manage professional profile + services, manage bookings, message, notifications, settings.
 - Admin:** moderation, analytics, users, submissions, bookings, verification, settings.
- Subscriptions/pricing flows via Stripe checkout initiation from the frontend.

Primary routing and flows are defined in `src/App.tsx`. Role and verification gating is enforced in `src/components/ProtectedRoute.tsx`.

Core user personas

- Talent** (role: "talent") — applies to casting calls and participates in auditions/livestreams.
- Casting Director** (role: "casting_director") — posts casting calls, manages applicants and pipeline.
- Industry Professional** (role: "industry_professional") — sells services, receives bookings, manages business profile.
- Admin** (role: "admin") — moderation + analytics + operational management.

Critical business objectives (frontend measurable)

- Conversion to signup and subscription (pricing → join → checkout initiation).
- High-quality posting + applying funnels (director posts; talent applies).
- Retention through messaging/notifications and dashboard usefulness.
- Trust & safety posture via verification gating UX (redirects, pending states).

Mission-critical features

- Authentication UI + session persistence (`localStorage` token/userData) via `src/contexts/AuthContext.tsx`.
- Role-based routing and verification gating via `src/components/ProtectedRoute.tsx`.
- Pricing display and checkout session creation via `src/pages/Pricing.tsx` and `src/lib/api.ts`.
- Real-time and livestream experiences: `Socket.IO` wrapper `src/lib/socket.ts` and `Agora` usage in `src/pages/talent/Livestream.tsx`.

2. Testing Scope

Frontend Testing

UI Rendering

- All routed pages render without crashes (smoke tests derived from routes in `src/App.tsx`).
- Loading placeholders render while data is pending (spinners, skeletons, empty placeholders).

Responsive Design

- Dashboard layouts (Talent/Director/Professional/Admin) across mobile, tablet, desktop breakpoints.
- Complex tables/grids (applicants pipeline, admin tables) remain usable on narrow widths.

Browser Compatibility

- Chrome/Edge/Safari coverage (WebRTC/livestream and upload behaviors especially).
- Verify `import.meta.env` usage works across target browsers.

Accessibility

- Keyboard navigation in tabs/menus/dialogs/selects (Radix UI components).
- Focus management in modals and dropdowns.

Navigation

- Public → join/sign-in → role dashboard.
- Role mismatch redirects (ProtectedRoute role map).

Forms

- Auth forms (sign-up/sign-in/forgot/reset/verify).
- Casting creation/edit forms (director).
- Profile/service forms (talent/professional/director).

Error / Loading / Empty States

- API failures show toasts and safe fallbacks.
- Empty results show purposeful UX (no applicants/bookings/invoices/notifications).

Integration Testing (Frontend-side)

- Stripe checkout redirection entry points: `src/pages/Pricing.tsx` and `src/pages/CheckoutPage.tsx`.
- Stripe JS bootstrapping: `src/lib/stripe.ts`.
- Socket connection lifecycle: `src/lib/socket.ts`.
- Agora livestream join/leave behavior: `src/pages/talent/Livestream.tsx`.

Security Testing (Frontend-relevant)

- Route protection + verification gating correctness (no route loops, no unauthorized screen flash).
- No sensitive data exposure in UI/logs (tokens never printed).
- Input validation UX (required fields, length constraints, file types/sizes).
- XSS hygiene: verify no unsafe HTML injection patterns are used.

Performance Testing (Frontend)

- Route lazy loading works (safeLazy usage) and avoids blank screens.
- Large lists remain responsive (browse talent, applicants, admin tables).
- Livestream CPU/network load checks on low-end devices.

Mobile Testing

- Touch interactions on tabs, dialogs, selects, tables.
- Livestream UX on mobile Safari/Chrome (permissions prompts and fallback behavior).

3. Complete User Journey Testing Matrix

User Journey	Steps	Expected Outcome	Priority
Public landing → Pricing	/ → /pricing	Pricing renders categories, billing toggle, correct plan cards	P0
Pricing → Join (logged out)	On /pricing click "Get Started" on a paid plan	Navigates to /join/:type?plan=...&cycle=...	P0
Pricing → Checkout (logged in)	Logged in → click "Get Started"	Creates checkout session → redirects to returned URL	P0
Enterprise custom path	Select Enterprise ("Custom") plan	"Contact Sales" (mailto) and no checkout call	P0
Sign up	/join/:type submit	Account created or clear error shown	P0
Sign in	/sign-in submit	Session stored; redirects to correct role dashboard	P0
Email verification	/verify-email with token	Verified state displayed; gating lifted	P0
Verification gating	Unverified non-admin user visits a gated route	Redirects to /verification-pending except allowed paths	P0
Talent: browse castings	/talent/browse-cast → open detail	List and detail render; empty state handled	P0
Talent: submit audition	/talent/browse-cast/:id/submit	Form validates and submits; success feedback	P0
Director: create/edit casting	/director/create and /director/projects/:id/edit	Form loads/saves; errors handled	P0
Director: applicants pipeline	/director/applicants	Stages/actions work; UI stays consistent	P0
Messaging (all roles)	Open messages route → send a message	Threads load; send works; failures show toast	P0
Livestream join	/talent/livestream/:id (or public route)	Permissions handled; join/leave cleanly	P0
Admin: users management	/admin/users actions	Actions show confirmation/toast and refresh	P0

4. Feature-by-Feature Test Cases

A) Routing, Auth, Role & Verification Gate

References: src/App.tsx, src/components/ProtectedRoute.tsx, src/contexts/AuthContext.tsx

Happy Path

- **Authenticated user enters own dashboard:** sign in as talent → visit /talent → dashboard loads with no redirect.
- **Role redirect for wrong dashboard:** sign in as professional → visit /director → redirects to /professional.

Edge Cases

- **Refresh on protected route:** sign in → hard refresh on /director/projects → brief loader → route resolves without loop.
- **Verification exceptions:** unverified non-admin user → profile routes allowed; other routes redirect to /verification-pending.

Negative

- **Unauthenticated user hits protected route:** clear storage → visit /admin/users → redirected to /sign-in.

Regression

- New routes added under dashboards inherit correct gating.

B) Pricing & Subscriptions

References: src/pages/Pricing.tsx, src/config/subscriptionPlans.ts, src/lib/api.ts

Happy Path

- **Correct plan prices display:** verify all plans across monthly/yearly toggle.
- **Logged-out subscribe routes to join:** click "Get Started" → navigates to join route with plan/cycle query params.

Edge Cases

- **Plans fetched from API override local config:** UI renders without missing-field crashes.

Negative

- **Checkout session API failure:** show toast error; button state resets.

Regression

- Plan keys remain stable end-to-end: pricing → join → checkout session payload.

C) Checkout Selection Page

References: src/pages/CheckoutPage.tsx

Happy Path

- **Select plan and cycle updates price:** switching between monthly/yearly updates the displayed price.
- **Logged-in user proceeds to checkout:** creates checkout session and redirects.

Edge Cases

- **Enterprise custom routing:** selecting Enterprise routes to sales mailto; no checkout call.

Negative

- **No plan available (defensive):** shows error and blocks submit.

D) Profiles (Public + Unified Profile)

References: `src/contexts/ProfileContext.tsx`, `src/pages/UnifiedProfile.tsx`, public profile routes in `src/App.tsx`

Happy Path

- **View public profile:** visit `/profile/:id` and role-specific profile routes.
- **Edit own profile:** toggle edit → save → correct role-specific update endpoint is called.

Negative

- **Profile not found:** error state shown and navigation fallback works.

E) Casting Calls & Applications

References: public casting routes and talent submit routes in `src/App.tsx`

Happy Path

- **Public browse casting list and detail:** `/browse-cast` → `/cast/:id` renders correctly.
- **Talent submit audition:** submit form validates and completes successfully.

Edge Cases

- **Casting closed state:** CTA disabled or informative message displayed.

Negative

- **Missing required fields:** inline validation appears; no API call.

F) Director: Create/Edit Casting

Reference: `src/pages/director/CreateCasting.tsx`

Happy Path

- **Create new casting call:** complete required fields → publish → success toast and expected navigation.
- **Edit existing casting:** load by id → update → save persists changes.

Negative

- **Unauthorized edit attempt:** backend denies → UI shows error and does not silently fail.

G) Director: Applicants Management Pipeline

Reference: `src/pages/director/ApplicantsManagement.tsx`

Happy Path

- **Filter by project and stage:** list updates correctly with counts.
- **Stage transitions:** moving candidates between stages updates UI consistently.

Edge Cases

- **High volume:** 200+ applicants remains usable (scroll, selection, actions).

Negative

- **Action failure:** toast error and UI reverts or refetches to avoid incorrect state.

H) Messaging & Notifications

References: `src/lib/socket.ts`, messaging endpoints in `src/lib/api.ts`

Happy Path

- **Load conversations:** list renders; selecting a thread shows messages.
- **Send message:** message appears and persists after refresh.

Edge Cases

- **Reconnect:** network drop → reconnect → messaging recovers without duplication.

Negative

- **Expired auth:** 401 clears token; protected routes redirect to sign-in.

I) Livestream / Auditions (Agora)

Reference: `src/pages/talent/Livestream.tsx`

Happy Path

- **Join with camera/mic allowed:** local preview shows; remote participants visible; leave is clean.
- **Mute/unmute:** toggles reflect real track state.

Edge Cases

- **Permission denied:** clear UX for retry or join with limited capabilities.
- **Mobile Safari constraints:** stable join or graceful fallback.

Negative

- **Invalid livestream id:** error state shown; user can navigate back.

J) Admin Console

Reference: admin routes in `src/App.tsx`

Happy Path

- **Users management actions:** suspend/verify/delete show confirmation/toast and refresh.
- **Analytics render:** charts render and handle empty states.

Negative

- **Forbidden access:** non-admin visiting `/admin` redirects to their dashboard.

5. API Testing Checklist (Frontend Dependencies)

Frontend contract expectations:

- Successful responses typically follow `{ success: true, data: ... }`.
- 401 clears token via axios interceptor (frontend should handle resulting redirects gracefully).

Key endpoint groups exercised by the frontend

- **Auth:** `register/login/me/verify/forgot/reset/logout`.
- **Subscriptions:** `plans, create-checkout-session, status, quota, payment methods, invoices, customer portal`.
- **Casting:** `casting-calls CRUD, close, my listings`.
- **Applications:** `submit, list, shortlist/reject/accept, communication, withdraw`.
- **Profiles/Services:** `get/update profiles, services CRUD and stats, uploads`.
- **Messaging/Notifications:** `conversations, messages, notifications read states`.
- **Livestream:** `create/join/start/end/participants/messages/invite/cohost`.

- **Admin:** users, moderation, analytics, submissions, bookings, verifications, settings.

QA Focus Risk: Ensure the invoices endpoint used by the frontend maps to the correct backend route, and that billing history UIs do not accidentally render payment-method data.

6. Database Testing Checklist (Frontend Interpretation)

- **Session storage integrity:** token and userData stored/cleared correctly.
- **React Query cache correctness:** query keys, invalidation, and stale UI after mutations.
- **Duplicate prevention:** prevent double submits and duplicate UI rows after retries.
- **Deep-link stability:** route changes should not break shareable links.

7. Critical Risk Assessment

Critical

- **Payments/Subscriptions:** checkout creation + redirect reliability; Enterprise must never attempt checkout.
- **Role/verification gating:** incorrect gating blocks legitimate users or exposes admin/director screens.
- **Livestream stability:** permissions and WebRTC handling across devices/browsers.

High

- **Applicants pipeline correctness:** stage changes and bulk actions can desync UI vs server.
- **Messaging reliability:** disconnects, duplicates, ordering issues.

Medium

- Large list performance (browse/search, admin tables, applicants lists).
- File upload UX (large files, invalid types, retries).

Low

- Static pages rendering (Terms/Privacy/Guides/FAQs).

8. Pre-Launch Testing Plan

Must Pass Before Launch (P0)

- Auth: sign-in/sign-up/forgot/reset/verify-email flows.
- Role routing: correct dashboard gating and redirects.
- Pricing correctness: all plan prices render correctly; billing toggle works.
- Subscription initiation: logged-in user can initiate checkout; Enterprise routes to sales.
- Director create/edit casting works and is stable.
- Talent can browse casting and submit audition without crashes.
- Messaging basic send/receive works (REST at minimum; socket recommended).
- Livestream join/leave + permission handling works on key browsers.

Recommended Before Launch (P1)

- Applicants pipeline actions + filters stable under load.
- Professional services CRUD and bookings views stable.
- Admin user operations are safe (confirmations, toasts, refreshes).

Nice To Have (P2)

- Cross-browser sweep including Safari.
- Performance profiling of the largest pages (livestream, applicants, profiles).
- Accessibility audit of dialogs, tabs, and selects.

9. Manual QA Checklist

- [] Public pages load: /, /about, /contact, /news, /privacy, /terms
- [] Pricing shows correct plan prices (monthly + yearly)
- [] Enterprise shows “Contact Sales” and does not attempt checkout
- [] Sign up works for each role via /join/:type
- [] Sign in persists session across refresh
- [] Unauthenticated access to dashboards redirects to /sign-in
- [] Unverified user is redirected to /verification-pending when required
- [] Talent can browse castings and open casting detail
- [] Talent can submit an audition (happy path)
- [] Talent submissions list updates after submit
- [] Director can create a casting call
- [] Director can edit a casting call
- [] Director applicants pipeline loads and filters
- [] Director can change applicant stage (shortlist/reject/etc.)
- [] Professional can create/edit services
- [] Professional bookings page loads and opens booking details
- [] Messaging pages load and can send a message
- [] Notifications page loads and can mark notifications read
- [] Livestream: permissions prompt works; can join and leave cleanly
- [] Admin: users page loads; suspend/verify/delete actions behave correctly
- [] NotFound route shows for invalid paths

10. Automated Testing Recommendations

Unit + Component Tests (P0)

- Vitest + Testing Library: ProtectedRoute redirects and verification exceptions.
- Pricing plan rendering: monthly/yearly switch and Custom handling.
- Checkout selection logic: logged out routes to join, logged in calls checkout session.

Integration Tests (P1)

- Critical “API failure → toast + retry enabled” tests for pricing checkout, casting save, applicants actions.

E2E Tests (P0)

- Playwright: signup/login per role, pricing→join, pricing→checkout initiation, director create casting, talent submit audition.

Smoke/Regression Suite (P0)

- Minimal route smoke tests that visit each route from src/App.tsx with role-appropriate auth stubs.

11. Bug Reporting Template

```
Title:
Severity (Blocker/Critical/Major/Minor):
Environment (Browser/OS/Device):
Build/Commit:
User Role (talent/director/professional/admin):
Preconditions (logged in? verified?):
Steps to Reproduce:
Expected Result:
Actual Result:
Screenshots/Video:
```


Console Logs:
Network Logs (failed request + response):
Notes/Impact:
Priority (P0-P3):

12. Launch Readiness Score (Frontend)

- **Overall MVP Frontend Readiness:** 72 / 100
- **Testing Coverage (current):** 35 / 100
- **Security (frontend-side):** 70 / 100
- **Performance:** 65 / 100
- **Reliability:** 75 / 100

Key focus before launch: stabilize payments/checkout flows, validate route gating, and run livestream and applicants pipeline stress on multiple devices/browsers.